Introduction to Blockchain

A distributed ledger technology that enables secure, transparent, and decentralized transactions

Decentralized Security

No single point of failure; distributed ledger with cryptographic protection ensures data integrity and resilience against attacks.

Transparency

All validated transactions are publicly visible and verifiable by every participant, fostering trust and accountability.

Immutable Chain

Blocks are cryptographically linked, creating an unchangeable historical record where data cannot be altered or deleted.

Consensus Mechanism

Network participants agree on transaction validity through protocols like PoW or PoS, ensuring ledger integrity and preventing fraud.

Smart Contracts

Self-executing contracts with terms directly written into code

Smart contracts are **computer programs** that automatically execute, control, or document events and actions according to predefined terms. They run on blockchain networks, eliminating intermediaries and ensuring **trustless execution** of agreements.

Coined by Nick Szabo in 1994, they are like digital vending machines for agreements: once conditions are met, actions are automatically triggered without human intervention.

Key Benefits

- Automated Execution: Eliminates manual processes and human error.
- Trustless & Secure: Inherits blockchain security; execution guaranteed by code.
- Cost Efficient: Reduces transaction costs by removing intermediaries.
- Transparent & Immutable: Publicly verifiable and unchangeable records.
- Global Accessibility: Accessible to anyone with internet and blockchain access.

Solidity Programming Language

Object-oriented language for implementing smart contracts on Ethereum

What is Solidity?

Statically typed, contract-oriented programming language designed for developing smart contracts that run on the Ethereum Virtual Machine (EVM). Syntax influenced by C++, Python, and JavaScript.

Key Features

- Inheritance & Libraries: Code reusability and modularity.
- Complex User-Defined Types: Organize data with structs and enums.
- Static Typing: Catches errors at compile time for robust contracts.
- JavaScript-like Syntax: Easier transition for web developers.

Use Cases

DeFi protocols, NFT marketplaces, DAOs, token contracts (ERC-20, ERC-721), and various decentralized applications (dApps).

Simple Contract Example

```
// SPDX-License-Identifier: MIT pragma solidity
^0.8.0; contract SimpleStorage { uint256 private
storedData; // State variable, permanently stored
constructor(uint256 initialValue) { storedData =
initialValue; } function set(uint256 x) public {
storedData = x; } function get() public view returns
(uint256) { return storedData; } }
```

Hardhat Development Framework

Comprehensive Ethereum development environment for professionals

Hardhat is an **open-source Ethereum development environment** designed to streamline the entire workflow for smart contracts and dApps, offering a robust set of tools for compiling, testing, deploying, and debugging.

Development Workflow

Compile → Test → Deploy → Debug

Key Features

- Local Ethereum Network: Instant testing without real transaction costs.
- Advanced Debugging: Detailed Solidity stack traces and console.log.
- Rich Plugin System: Extensible with community-developed or custom plugins.
- Console.log Support: Print values directly within Solidity code.
- Mainnet Forking: Test against real-world data from live networks.
- TypeScript Support: Native support for robust and maintainable dApps.

Hardhat vs Truffle

Choosing the right Ethereum development framework

Hardhat

Modern & Flexible

- Built-in local Ethereum network for instant testing and rapid iteration.
- Advanced debugging with detailed Solidity stack traces.
- Console.log in Solidity for easier contract debugging.
- Flexible plugin architecture for custom functionalities and integrations.
- Mainnet forking capabilities to test against real-world data.
- Native TypeScript support for robust and maintainable dApps.

Best For

New projects, complex debugging, modern development workflows, and TypeScript users.

Truffle

Established & Mature

- Comprehensive suite with Ganache for a complete development environment.
- Built-in smart contract compilation with dependency handling.
- Automated contract testing with JavaScript and Solidity-based tests.
- Scriptable deployment migrations for managing complex deployments.
- Interactive console for direct interaction with deployed contracts.
- Large community and extensive resources for learning and support.

Best For

Beginners, established workflows, enterprise projects, and those preferring an integrated solution.

MetaMask Wallet

Your gateway to the decentralized web

MetaMask is the most popular **non-custodial cryptocurrency wallet** and primary gateway to the decentralized web. It allows users to manage Ethereum accounts and interact with dApps directly from their browser, acting as a bridge between traditional web and blockchain.

100M+

Monthly Active Users

Available Platforms

Browser

Mobile

Desktop

Key Features

Private Key Management: Users retain full control over their private keys.

Token Swapping: Built-in feature to exchange cryptocurrencies.

dApp Integration: Seamlessly connects with thousands of dApps.

Multi-Network Support: Supports various EVM-compatible networks.

Secure Transactions: Review and approve transactions before broadcast.

NFT Support: View and manage non-fungible tokens directly.

Ethers.js for Web3

Lightweight library for interacting with the Ethereum blockchain

What is Ethers.js?

A **compact JavaScript library** providing a complete API for interacting with the Ethereum blockchain and its ecosystem. It abstracts complexities of JSON-RPC, cryptography, and data encoding.

Key Advantages

- Lightweight (77KB compressed) for efficient web apps.
- Modular architecture: Import only needed components.
- TypeScript support for type safety and maintainability.
- Better error handling with descriptive messages.
- ENS name resolution for human-readable addresses.

vs Web3.js

Ethers.js is often preferred for its modularity, lightweight nature, and improved developer experience, with clearer separation of concerns between providers, signers, and contracts.

Basic Usage Example

```
// Import ethers import { ethers } from 'ethers'; // Connect
to Ethereum network const provider = new
ethers.JsonRpcProvider(
'https://mainnet.infura.io/v3/YOUR_KEY' ); // Replace
YOUR_KEY with actual Infura project ID // Get wallet balance
const balance = await provider.getBalance(
'0x742d35Cc6634C0532925a3b8D' ); // Returns balance in Wei //
Convert to ETH and log const ethBalance =
ethers.formatEther(balance); console.log(`Balance:
${ethBalance} ETH`); // Interact with smart contract const
contract = new ethers.Contract( contractAddress, abi,
provider ); // Requires contract address and ABI const result
= await contract.someMethod(); // Call a contract method
```